

Optimización del problema “Subir Escaleras”: de recursión exponencial a fórmula cerrada (Binet)

Participantes: Jhon Perlaza, Heidi Sofía Barrera, Andrés Felipe Cadena, Juan Ahumada

29 de mayo de 2026

Abstract

Este trabajo estudia el problema clásico de contar cuántas formas existen de subir n escalones utilizando pasos de tamaño 1 o 2. Se implementa una solución recursiva ingenua sin memoización, cuyo tiempo crece exponencialmente, y se compara contra una implementación optimizada basada en la relación con Fibonacci y la fórmula cerrada de Binet. Se presenta un análisis de complejidad y un benchmarking que evidencia la diferencia de escalabilidad a medida que n aumenta.

1 Introducción

Definimos $W(n)$ como el número de formas de subir n escalones cuando se permiten saltos de 1 o 2. Cada forma termina con un paso de 1 desde $n-1$ o un paso de 2 desde $n-2$, por lo que se cumple:

$$W(n) = W(n-1) + W(n-2), \quad W(0) = 1, \quad W(1) = 1. \quad (1)$$

Esta recurrencia conecta el problema con Fibonacci y permite obtener mejoras drásticas en complejidad temporal.

2 Desarrollo matemático

Sea $F(n)$ la sucesión de Fibonacci con $F(0) = 0$ y $F(1) = 1$. Se verifica (por inducción) que:

$$W(n) = F(n+1). \quad (2)$$

2.1 Implementación pesada: recursión ingenua

La implementación pesada calcula $W(n)$ aplicando recursión directa sin guardar resultados intermedios. Si $T(n)$ representa el tiempo (número de llamadas), entonces:

$$T(n) = T(n-1) + T(n-2) + O(1). \quad (3)$$

Esta recurrencia crece exponencialmente y se tiene:

$$T(n) = \Theta(\varphi^n), \quad (4)$$

donde $\varphi = \frac{1+\sqrt{5}}{2} \approx 1.618$ es la proporción áurea. Por tanto, **no** es $\tilde{O}(n^2)$: el árbol de recursión se expande de forma explosiva.

2.2 Implementación optimizada: fórmula cerrada (Binet)

Usando la fórmula de Binet:

$$F(k) = \frac{\varphi^k - \psi^k}{\sqrt{5}}, \quad \psi = \frac{1-\sqrt{5}}{2}, \quad (5)$$

y como $W(n) = F(n+1)$, puede calcularse el resultado mediante una expresión cerrada.

Discusión de complejidad. En un modelo ideal (evaluar una fórmula sin iterar hasta n), esta aproximación puede describirse como $O(1)$ en pasos matemáticos. En una implementación exacta, el costo real depende del tamaño del número (cantidad de dígitos) para n grande. Aun así, la mejora frente a $\Theta(\varphi^n)$ es enorme y se evidencia empíricamente.

3 Resultados experimentales (Benchmark)

Se implementó un script de benchmarking en Python que compara:

- **Pesada (exponencial):** recursión ingenua sin memoización.
- **Optimizada (“O(1)”):** Binet híbrida (usa `float` para n pequeño y `decimal` para n grande).

Para evitar tiempos excesivos, la versión exponencial se omite para $n > 35$. El benchmark genera los archivos en la carpeta `results/`:

- `benchmark.csv`
- `benchmark.png` (escala log-log)
- `benchmark_small.png` (zoom para $n \leq 35$)

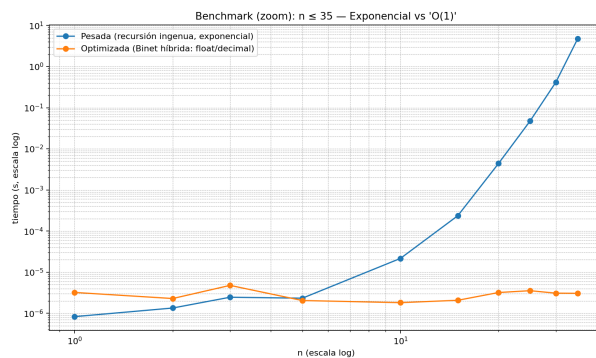


Figure 1: Comparación en escala log-log para $n \leq 35$ (ambos algoritmos medidos).

4 Aplicaciones reales

La reducción de complejidad mediante matemáticas, precomputación o estructuras eficientes aparece en:

- **Bases de datos:** índices (B-trees, hashing) aceleran búsquedas de escaneo lineal a $O(\log n)$ u $O(1)$ promedio.
- **Criptografía:** técnicas como exponenciación rápida reducen costos de $O(n)$ a $O(\log n)$.
- **Programación dinámica:** memoización transforma recursiones exponenciales en soluciones polinómicas.

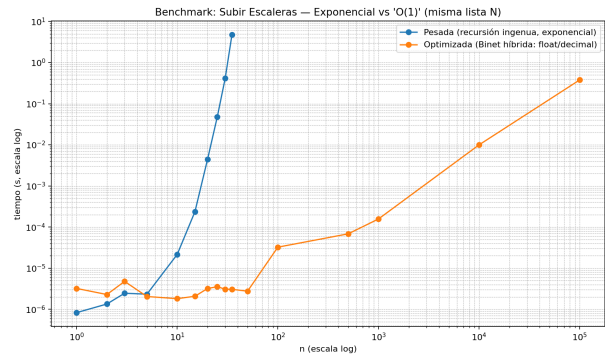


Figure 2: Comparación log-log para la lista completa. El exponencial aparece solo hasta $n = 35$ (luego se omite).

5 Repositorio

Código fuente ():

<https://github.com/JhonPerlaza0614/Entrega-Final.git>

6 Referencias

- E. W. Weisstein, “Fibonacci Number”, *MathWorld*. <https://mathworld.wolfram.com/FibonacciNumber.html>
- Wikipedia, “Fibonacci number” y “Binet’s formula”. https://en.wikipedia.org/wiki/Fibonacci_number